CMI Sensor Project — Team Guide (Preprocessing)

Oct 9, 2026 · @Ameen

How to read this guide

We two own **data preprocessing** for our SC4000 group project on the Kaggle competition *CMI – Detect Behavior with Sensor Data*. This guide starts from zero: no machine learning or sensor background is assumed. Read it top to bottom once (about 30 minutes) and you will know what the project is, what the data looks like, and exactly what our part is and why each step exists.

- **Sections 1–3** explain the idea: the course, the competition, the scoring.
- **Sections 4–6b** explain the data: what is in it, what is missing, what makes it tricky, and what the top teams learned.
- **Sections 7–9** are our actual job: every preprocessing step, feature and augmentation, with the reason for it and how we will prove it helps.
- **Section 10** is the plan and timeline. **Section 11** lists facts we still must confirm by running the data. **Section 12** is a glossary — jump there whenever a word is unfamiliar. Sources are at the very end.

Facts about the dataset come from the competition's description. Numbers like row counts are approximate until we run our audit script (`eda/01_data_audit.py`) on the real files; anything still unconfirmed is listed in Section 12.

1. The course project in one page

The project is worth **35% of the module**, is due **13 Nov 2026, 11:59 pm** on NTULearn, and is marked mostly on how well our model ranks and how convincingly we justify it.

What we hand in	Weight	Notes
Project report	30%	AAAI template, max 10 pages including references. Breaking the format = 50% penalty.
Presentation video	5%	Max 10 minutes, every member must speak, private YouTube link in a .txt file.

What we hand in	Weight	Notes
Kaggle submission (.ipynb / .csv)	not graded	Only used to check our reported score is real.

How the report is graded (most important first):

1. **Convincingness** — do we explain *why* each step exists, and do experiments prove it helps?
2. **Leaderboard performance** — our rank on the Kaggle **public** leaderboard, in bands: top 10%, 30%, 50%, 70%, rest.
3. **Novelty** and **writing quality** — new ideas, clear and well-organised text.

Convincingness and leaderboard count equally and more than the other two. Preprocessing feeds both: good preprocessing raises the score, and well-explained preprocessing with experiments is exactly what "convincing" means.

Rules we must not break

- We may *read* public Kaggle notebooks for ideas, but must not copy them. Copying = plagiarism.
- Any code we used as a reference must be acknowledged in the report.
- Do not publish our code publicly (keep the GitHub repo private). Avoid opening other SC4000 students' public repos for this competition, so our work stays clearly our own.
- Keep chat history and git commits: the report must state each member's contribution, and "lazy" members are graded differently.

2. The idea behind the competition

We must look at a short recording from a wrist sensor and say **which of 18 hand gestures** the person was doing — with special focus on spotting harmful habits called BFRBs.

What is a BFRB? A *body-focused repetitive behaviour* is a habit like pulling your hair, picking skin, or scratching — done over and over, often without noticing. It is linked to anxiety and OCD. If a smartwatch could detect these moments automatically, it could alert the person or help therapists track progress.

The device. The Child Mind Institute (CMI) built a wrist device called **Helios**, worn like a watch. It has three kinds of sensors (explained in Section 4): a motion sensor (IMU), 5 temperature sensors, and 5 distance cameras.

How the data was recorded. Each volunteer wore Helios and was told to do a gesture. One attempt = one **sequence**, a short clip of sensor readings (roughly 10 readings per second, a few seconds long). Every sequence follows the same script:

1. **Transition** — move the hand from resting to near the target body part.
2. **Pause** — hold still briefly.
3. **Gesture** — do the actual action (e.g. pull hair above the ear).

They repeated this in 4 body positions: sitting straight, sitting leaning forward, lying on the back, lying on the side.

The 18 gestures. 8 are BFRBs ("Target") and 10 are normal look-alike actions ("Non-target") chosen to be confusing — e.g. *scratching your neck* (BFRB) vs *scratching your knee* (normal).

Target — BFRB (8)	Non-target — normal (10)
Above ear – pull hair	Drink from bottle/cup
Forehead – pull hairline	Glasses on/off
Forehead – scratch	Pull air toward your face
Eyebrow – pull hair	Pinch knee/leg skin
Eyelash – pull hair	Scratch knee/leg skin
Neck – pinch skin	Write name on leg
Neck – scratch	Text on phone
Cheek – pinch skin	Feel around in tray and pull out an object
	Write name in air
	Wave hello

What we predict. For each test sequence, one gesture name. That is a **classification** problem on **time-series** data (data that is a sequence of readings over time, not a single row).

3. How we are scored

The score (0 to 1, higher is better) is the **average of two F1 scores**: one for "is it a BFRB or not?" and one for "which BFRB is it?".

$$\text{Score} = \tfrac{1}{2}\,\text{Binary F1} + \tfrac{1}{2}\,\text{Macro F1}$$

- **F1 score** is a balance between *precision* (when we say X, how often are we right?) and *recall* (of all real X's, how many did we catch?). 1.0 is perfect.

- **Binary F1** — we only check BFRB vs non-BFRB. Any of the 8 BFRB names counts as "BFRB".
- **Macro F1** — we check 9 classes: each of the 8 BFRBs separately, plus *one* lumped class "non-target" for all 10 normal gestures. "Macro" means each class's F1 is averaged equally, so rare classes matter as much as common ones.

What this means in practice

If the true gesture is…	and we predict…	Penalty?
Wave hello	Text on phone	None — both are non-target
Neck – scratch	Neck – pinch skin	Hurts Macro F1 (wrong BFRB), Binary still correct
Neck – scratch	Scratch knee/leg skin	Hurts both — the worst kind of mistake

So our effort should go into (1) cleanly separating BFRB from normal and (2) telling the 8 BFRBs apart. Confusing two normal gestures costs nothing. Top teams reported Binary F1 above 0.99 on validation, so in practice telling the 8 BFRBs apart decides the ranking. That is why fine details like hand position (ToF regions, gravity direction) matter.

4. The dataset: what is in it

The training data is one big table where **each row is one moment in time** and **each sequence is a group of rows** sharing a `sequence_id`. Roughly 575k rows, ~8,150 sequences, ~81 people, ~340 columns (to be confirmed by our audit).

The files

File	What it holds
`train.csv`	All sensor readings + the answers (gesture labels). This is what we learn from.
`train_demographics.csv`	One row per person: age, sex, handedness, height, arm length.
`test.csv`, `test_demographics.csv`	A tiny example of what the test data looks like. The real test set is hidden.

File	What it holds
`kaggle_evaluation/`	Kaggle's code that feeds our notebook one test sequence at a time.

How one sequence looks (simplified)

```
sequence_id,sequence_counter,subject,acc_x,acc_y,acc_z,rot_w,rot_x,rot_y,rot_
z,thm_1,...,tof_5_v63,gesture
SEQ_000007,0,SUBJ_059520,6.68,6.21,3.49,0.13,0.49,-0.74,-0.44,28.9,...,-1,Che
ek - pinch skin
SEQ_000007,1,SUBJ_059520,6.95,6.21,3.13,0.14,0.49,-0.74,-0.44,29.3,...,-1,Che
ek - pinch skin
SEQ_000007,2,SUBJ_059520,5.72,5.41,3.86,0.14,0.49,-0.74,-0.44,29.7,...,112,Ch
eek - pinch skin
```

(Values are illustrative.) `sequence_counter` is the time step: 0, 1, 2, … The label is the same on every row of a sequence.

The column groups, sensor by sensor

1. Accelerometer — `acc_x`, `acc_y`, `acc_z` (3 columns). Measures how hard the wrist is being pushed in each of 3 directions, in m/s². Important catch: it **always includes gravity** (~9.8 m/s²). Even a perfectly still hand reads about 9.8 split across x, y, z depending on how the wrist is tilted.

2. Rotation — `rot_w`, `rot_x`, `rot_y`, `rot_z` (4 columns). Which way the wrist is facing in 3D, stored as a **quaternion** — 4 numbers that together describe a 3D rotation without the glitches that angles have. You do not need the maths; just know it tells us the watch's tilt and twist at each moment, and lets us undo the tilt (Section 7, step 5).

Together, accelerometer + rotation are called the **IMU** (Inertial Measurement Unit) — the same motion chip in every phone.

3. Thermopiles — `thm_1` … `thm_5` (5 columns). Five small non-contact thermometers around the watch, reading temperature in °C. When the hand is close to skin (face, neck), they read warmer. Useful to tell "hand near head" from "hand in the air".

4. Time-of-Flight (ToF) — `tof_1_v0` … `tof_5_v63` (320 columns). Five tiny distance cameras. Each takes an **8×8 = 64-pixel** picture where every pixel is a distance (roughly 0–250, larger = farther). So each sensor = 64 columns, ×5 sensors = 320 columns. A pixel value of **-1 means "nothing detected"** (no object in range) — not a real distance.

5. ID and label columns.

Column	Meaning	In test?
`row_id`, `sequence_id`, `sequence_counter`	Which row, which sequence, which time step	Yes
`subject`	Which person (links to demographics)	Yes
`gesture`	**The answer** — one of 18 gestures	No (we predict it)
`sequence_type`	Target (BFRB) or Non-Target	No
`behavior`	Which part of the script this row is in (e.g. moves hand to target, performs gesture)	No
`phase`	Transition or Gesture	No
`orientation`	Body position (sitting, lying, …)	No

6. Demographics (per person). `adult_child`, `age`, `sex`, `handedness`, `height_cm`, `shoulder_to_wrist_cm`, `elbow_to_wrist_cm`. Handedness matters most: the watch goes on the dominant wrist, so left-handers produce mirror-image motion.

5. What is there and what is NOT there

The training data has extra helper columns and full sensors; the test data does not — and about **half the test sequences have only the motion sensor**. Our pipeline must be built for the test reality, not the training luxury.

	Training data	Hidden test data
IMU (acc + rot)	Yes	Yes, every sequence
Thermopiles + ToF	Yes (some gaps)	**Only ~50% of sequences**; the rest are completely blank
`subject` + demographics	Yes	Yes
`gesture` (answer)	Yes	No — this is what we predict
`sequence_type`, `behavior`, `phase`, `orientation`	Yes	**No**
Many sequences at once	Yes	No — Kaggle gives our code **one sequence at a time**

Why this matters for us

- **Never use test-missing columns as inputs.** If a model learns from `orientation` or `phase`, it will have nothing to look at during testing and break. We *can* still use them during training as extra targets or for analysis (e.g. to find where the gesture is).
- **IMU-only is the main case, not an edge case.** The model must score well with just 7 motion columns. Thermopile + ToF are a bonus when present.
- **One sequence at a time.** Any preprocessing we write must work on a single sequence alone. Anything that needs "all data" (like the average used for scaling) must be computed once on training data, **saved to a file**, and loaded in the Kaggle notebook.
- **Code competition.** We do not upload a CSV of answers. We submit a Kaggle notebook containing our model and a `predict()` function. Our preprocessing code literally runs inside that function.

Missing values we already expect

Where	How it shows up	What it means
ToF pixels	`-1`	Sensor saw nothing in range — a real reading, not an error
ToF / thermopile whole sequence	All empty (NaN)	Sensor switched off — same as the IMU-only test half
ToF / thermopile a few rows	Some NaN	Sensor dropout / glitch

Where	How it shows up	What it means
Rotation	Some NaN rows	Orientation chip glitch

6. What makes this data tricky

Eleven problems in the raw data each map to a preprocessing step in Section 7; this is the "why" behind our whole job. P8–P11 come from our research (Section 6b).

#	Problem	Simple explanation	Fixed by step
P1	Gravity hides the motion	The accelerometer reads ~9.8 even when still. Lying down vs sitting changes the raw numbers a lot for the *same* gesture.	5
P2	Left vs right hand	Left-handers wear the watch on the left wrist, so their movements and sensor positions come out mirrored.	4
P3	Different lengths	Sequences range from ~29 to ~700 rows. Most models need equal-size inputs.	10
P4	Gesture is only at the end	The first part of each sequence is just moving into position. The useful action is in the last part.	10
P5	Missing sensors	Half the test set has no thermopile/ToF; training has gaps and `-1` pixels.	3, 8
P6	Different scales	Acceleration ~0–20, temperature ~25–35, ToF ~0–254. Big-number columns can dominate a model.	11
P7	Same person in train and validation	Everyone moves in their own style. If one person is in both, our validation score is falsely high.	12
P8	Upside-down devices	Two subjects wore the watch rotated 180°, so all their axes and sensor positions are wrong.	2

#	Problem	Simple explanation	Fixed by step
P9	Quaternion sign jumps	A rotation can be written as q or −q. The numbers can flip sign mid-sequence although the wrist did not move, creating fake spikes.	3
P10	Rotation missing entirely	Some sequences have no rotation data, so gravity cannot be removed the normal way.	5 (fallback)
P11	Small dataset	~8,150 sequences from 81 people: a model can memorise people instead of learning gestures.	Section 8b

Two smaller issues: the 18 classes may not have equal numbers of sequences (class imbalance — checked in the audit), and 320 ToF columns is a lot of raw input that may be better summarised (step 7).

6b. What the top teams did (research, Oct 2026)

The competition ended on 2 Sep 2025 with 2,657 teams; the winners scored about 0.89. We studied the public write-ups to see which preprocessing ideas actually moved the score, so our plan is built on evidence, not guesses. We take *ideas* only and write our own code, crediting each source in the report (Sources section at the end).

Finding	Who	What it means for us
Gravity removal was the single most effective step	7th place; also 1st, silver solutions	Keep step 4 as the core step
Two subjects wore the device upside down (`SUBJ_019262`, `SUBJ_045235`, rotated 180°). Their validation scores were very low.	Reported in discussion; fixed by 3rd and 6th	Correct their data, don't delete it. 7th place deleted them and called that a mistake
Left-handers converted to right-handed across IMU, thermopile and ToF (swap sensors 3 ↔ 5, flip some axes)	Nearly all top teams	Mirroring must cover *all* sensors, not just acceleration

Finding	Who	What it means for us
Some test sequences have no rotation data at all, not only no ToF/thermopile	2nd place built 4 models for this	We need a fallback when gravity removal is impossible
Quaternions jump (q and −q are the same rotation, so values can flip sign suddenly)	2nd place used a 6D rotation format; 3rd smoothed and added sign-flip augmentation	Make quaternions continuous before computing any rotation feature
Augmentation gave large gains because the data is small	7th, 3rd, 10th, 1st	Add a full augmentation section (8b)
Keep the **end** of the sequence: 7th place used the last 140 steps at test, random crops of 70–200 in training	7th	Confirms our keep-end rule and gives a length to start from
Sequence length: mean ~71 steps, min 29, max 700; a length of 128 keeps 95% of sequences whole	Academic paper on the same data (2026)	Starting length range for step 8
Per-timestep phase labels used as an extra training target (where is the gesture?)	2nd, 3rd, 6th	Prepare per-row phase labels as an output, never an input
Richer labels: gesture × orientation × initial state (up to 102 classes)	2nd, 4th	Prepare these combined labels for the modellers
Each subject does each gesture once per orientation (51 combos × 2 starts = 102 sequences each)	Shared by top 5	Enables a powerful post-processing trick (for the modelling team)
Demographics as features barely helped	Several teams	Use handedness for mirroring only; height/arm length is a low-priority test
Raw signals + simple derivatives beat frequency (FFT) features	Several teams	Skip spectrograms; focus on time-domain features

7. Our job: the preprocessing pipeline, step by step (v2)

Our job is to turn the raw table into clean, equal-sized, well-scaled inputs that **any** model (tree, CNN, GRU, 2D image model) can train on directly, and that work identically on one hidden test sequence inside Kaggle, whatever sensors it has. Every step says **what** we change, **how**, and **why**. Steps marked ★ are new after our research.

1. **Load, sort and sanity-check**
   - *What:* Sort rows by `sequence_id` then `sequence_counter`. Check for duplicates, counter gaps, one gesture and one subject per sequence. Flag broken sequences, e.g. ones with no Gesture-phase rows (1st place removed these from training).
   - *Why:* Every later step assumes clean, time-ordered sequences. Broken training examples teach the model wrong things.
2. **★ Fix the two upside-down subjects**
   - *What:* For `SUBJ_019262` and `SUBJ_045235`, rotate their data 180° around the watch's Z axis: flip the sign of `acc_x` and `acc_y`, apply the same 180° turn to the quaternion, and swap the thermopile/ToF sensors that trade places. Confirm with plots that their averages then match everyone else.
   - *Why:* Fixes P8. Their data looks like a different device, which confuses the model. 3rd and 6th place corrected them; deleting them (7th place) lost data.
   - *Stretch goal:* 6th place also detected upside-down wearers in the *test* set and corrected them there. We can try a simple check (e.g. average gravity direction pointing the "wrong" way).
3. **★ Clean the IMU signals**
   - *What:*
     - **Quaternion sign continuity:** walk through each sequence; if a quaternion points the opposite way from the previous one (their dot product is negative), flip its sign. Then rescale to length 1.
     - **Fill gaps:** short NaN gaps in acc/rot → straight-line interpolation between neighbours, then forward/back-fill at the edges.
     - **Flag sequences with no rotation at all:** `rot_missing = 1`.
   - *Why:* Fixes P9 and P10. A sign flip looks like a huge sudden twist that never happened, ruining angular-velocity features. Interpolation is more realistic than copying the last value for smooth motion.
4. **Mirror left-handed subjects (all sensors)**
   - *What:* For left-handed subjects: flip the sign of the sideways IMU axis and the matching quaternion parts; swap `thm_3` ↔ `thm_5` and `tof_3` ↔ `tof_5`; mirror each 8×8 ToF image left-right.

- *Why:* Fixes P2. The watch's left and right sensors trade places on the other wrist, so mirroring only the IMU would leave the ToF and thermopiles wrong. Sources disagree on exactly which axes to flip, so we confirm with plots of left vs right subjects (Section 11).

5. **Remove gravity, with a fallback**
   - *What:* Normal case: rotate acceleration into the room frame using the quaternion, subtract 9.81 from the vertical. ★ Fallback when `rot_missing = 1`: estimate gravity as the slowly changing part of the acceleration (a low-pass filter) and subtract it.
   - *Why:* Fixes P1, the most effective step according to 7th place. The fallback means sequences without rotation still get approximate motion features instead of none.
6. ★ **Light smoothing (for derivative features only)**
   - *What:* A small moving-average or low-pass filter on the IMU *before* computing jerk and angular velocity. Raw signals are kept too.
   - *Why:* "Change between rows" features amplify sensor noise. Smoothing first makes them meaningful. Tested in the ablation.
7. **Add engineered features** — full list and reasons in Section 8.
8. **Thermopile and ToF**
   - *What:* ToF `-1` → treat as "far" (or masked), scale valid values 0–254 to 0–1. Per sensor, per row: mean, min, max, std, valid fraction, plus ★ coarse regional averages (e.g. a 3×3 grid of the 8×8 image, idea from 1st place). Keep the full 8×8 images as a second output for image-style models. Missing flags `thm_missing`, `tof_missing`.
   - *Why:* Fixes P5. Summaries keep the meaning ("how close is the face, and on which side?") in far fewer columns.
9. ★ **Build all label types (training only)**
   - *What:* From the train-only columns, create: the 18-gesture label; the binary BFRB label; the 9-class scoring label; a **per-row phase label** (transition / pause / gesture); the orientation label; and a **combined label** gesture × orientation × initial state (how the sequence started).
   - *Why:* Top teams trained models to predict these as extra outputs, which teaches the model where the gesture is and how posture affects it. They are **outputs only**, never inputs, so there is no leakage.
10. **Make every sequence the same length**
    - *What:* Fixed length L chosen from the audit (start with 128, which keeps ~95% whole). Keep the **end**; pad at the start with zeros plus a **mask column** (1 = real row, 0 = padding). For tree models: per-sequence statistics over the whole sequence *and* over the last 25%.
    - *Why:* Fixes P3 and P4. The mask stops the model treating padding as "hand perfectly still".

11. **Scale the numbers**
    - *What:* Standardise each feature using training-fold statistics only; save them. ★ Also produce an unscaled version, since 1st and 9th place used no scaler and relied on the network's own normalisation layers. The ablation decides.
    - *Why:* Fixes P6 without leakage. Keeping both lets the modellers choose.
12. **Split by person, fixed once for the whole team**
    - *What:* 5 folds with StratifiedGroupKFold, grouped by `subject`, stratified by gesture, fixed seed. Save the fold assignment as a file everyone uses.
    - *Why:* Fixes P7. If every teammate uses the same folds, everyone's scores are comparable and can be ensembled honestly.
13. **Package it for any model and any sensor situation**
    - *What:* One `preprocess(sequence, demographics)` function that works in all 4 sensor situations (full; no thermopile/ToF; no rotation; neither). It outputs (a) a per-row feature array + mask for neural nets, (b) one summary row for tree models, (c) ToF images. Saved files (scaler, label maps, length L, folds) go to a private Kaggle Dataset.
    - ★ *Parity test:* a script that runs the training path and the Kaggle `predict()` path on the same sequences and checks the outputs are identical.
    - *Why:* One function serves every model the team tries, and the parity test catches silent train/test mismatches before they cost leaderboard score.

8. Feature catalogue: every new column and why

A *feature* is an extra column we calculate from the raw ones to make a pattern easier for the model to see. Each one below has a reason; any that does not improve validation score in our tests gets dropped.

Feature	How it's calculated (in words)	Why it should help
`linear_acc_x/y/z`	Acceleration rotated into room frame, gravity subtracted (step 5)	Real hand motion, same for every posture
`acc_mag`	Length of the acceleration vector: $\sqrt{(x^2 + y^2 + z^2)}$	One number for "how hard is the hand moving", independent of direction

Feature	How it's calculated (in words)	Why it should help
`linear_acc_mag`	Same, on gravity-removed acceleration	Pure movement intensity — scratching is fast and repetitive, pinching is slow
`jerk` (per axis + magnitude)	Change in (smoothed) acceleration from one row to the next	Picks up sudden starts/stops, e.g. a quick hair pull
`angular_velocity_x/y/z`	Rotation between two consecutive quaternions, turned into a rotation speed	Wrist *twisting* — pinching twists, scratching does not
`angular_distance`	Size of the rotation between consecutive rows (one number)	Overall "how much did the wrist turn" per step (silver-medal solution)
★ `rot6d_1..6`	Quaternion converted to a rotation matrix; keep its first two columns (6 numbers)	A smooth rotation format with no sign jumps (2nd place; Zhou et al. 2019)
`gravity_dir_x/y/z`	Which way "down" points from the watch's point of view	Hand pointing up (toward head) vs down (toward leg) — separates neck vs knee scratching
★ `acc_cos_sim`	Similarity of acceleration direction between consecutive rows	Repetitive back-and-forth (scratching) flips direction often (7th place)
`tof_k_mean/min/max/std` (k = 1–5)	Per sensor, summary of the valid pixels each row	"How close is the nearest surface?" — face/neck is close for BFRBs
★ `tof_k_region_1..9`	Average of each cell of a 3×3 grid over the 8×8 image	Keeps *where* the object is (left/right/up/down), not just how close (1st place idea)
`tof_k_valid_frac`	Fraction of pixels not `-1`	How much of the view is filled by a nearby object

Feature	How it's calculated (in words)	Why it should help
`thm_1..5`, `thm_mean`, `thm_diff`	Raw temperatures, their average, change since previous row	Hand approaching skin warms the sensors; top teams found raw thermopile values enough
`tof_missing`, `thm_missing`, ★ `rot_missing`	1 if that sensor is off for this sequence	Lets the model switch behaviour per sensor situation
★ `mask`	1 for real rows, 0 for padding	Model ignores padded rows
`time_frac`	Row position / sequence length (0 to 1)	Tells the model where in the clip it is (the gesture is near 1)

Note for tree models (LightGBM/XGBoost): these features are then summarised per sequence (mean, std, min, max, value in the last 25% of the sequence), giving one row per sequence. **Neural networks** (1D-CNN / GRU) take the per-row features directly after padding (step 8).

8b. Data augmentation (training only)

Augmentation means making slightly altered copies of training sequences on the fly, so the model sees more variety than the 81 people provide. It fixes P11 and was one of the biggest gains for top teams. It is applied **only during training**, never to validation or test data. Each one below is randomly switched on per sequence.

Augmentation	What it does (simply)	Real-world variation it imitates	Starting setting	Used by
Random crop	Take a random-length window ending near the end	Gestures start at different times	Length 70–200 rows	7th
Time stretch / warp	Play the sequence a bit faster or slower, smoothly	People gesture at different speeds	±10–20% speed	1st, 7th, 10th; Um et al. 2017
Time shift	Slide the sequence a few rows	Slightly different start time	±a few rows	1st

Augmentation	What it does (simply)	Real-world variation it imitates	Starting setting	Used by
Rotation around vertical	Turn the whole motion around the up-down axis	Person facing a different direction	±60°	3rd
Small wrist rotation	Tilt the watch frame slightly	Watch sits differently on each wrist	±7°	3rd
Magnitude warp / scaling	Make signals slightly stronger or weaker, smoothly	Bigger or smaller movements	±10%	7th; Um et al. 2017
Jitter	Add tiny random noise	Sensor noise	Small, vs. signal spread	7th
Quaternion sign flip	Randomly use −q instead of q	Makes the model robust to sign jumps	50%	3rd
Sensor dropout	Blank thermopile/ToF (and sometimes rotation) + set missing flags	The IMU-only half of the test set	~50%	silver; 7th
Mixup (same class)	Blend two sequences of the **same** gesture and orientation	More in-between examples without mixing labels	Blend weight from 0–1	2nd, 4th, 7th, 10th
★ Mirror copy	Alternative to step 4: skip converting left-handers and instead train on original + mirrored copies of everyone	Model learns both hands directly; ablation compares it with step 4 and with both	50%	Our idea; tested in ablation

Rules: rotation-type augmentations are applied to the *raw* acc and quaternion first, then all features (gravity removal etc.) are recomputed, so everything stays physically consistent. Settings above are starting points; we tune them with the ablation.

9. How we prove each step helps

We prove each preprocessing step with an **ablation**: train the same model with and without that one step, using the same 5-fold subject split, and report the change in score. This table is our main contribution to the report's "convincingness" mark.

Rules for a fair test

- Same model, same settings, same 5 folds (fixed random seed) for every row.
- Report the competition score (Section 3) as the mean ± spread across 5 folds.
- Also report the score on **IMU-only** validation (thermopile + ToF blanked out), since that is half the real test set.
- Change **one thing at a time**, adding steps cumulatively.

Ablation table we will fill in (scores come from our experiments)

Row	Setup	CV score (all sensors)	CV score (IMU-only)
A	Baseline: raw columns, NaN → 0, pad end, no scaling		
B	A + IMU cleaning: sign continuity, interpolation (step 3) + ToF/thermopile handling (step 8)		
C	B + keep-end length + mask (step 10) + scaling (step 11)		
D	C + upside-down fix (step 2)		
E	D + handedness mirroring, all sensors (step 4)		
F	E + gravity removal with fallback (step 5)		
G	F + smoothing + engineered features (steps 6–7, Section 8)		
H	G + basic augmentation: crop, stretch, jitter, sensor dropout (8b)		
I	H + rotation augmentation + same-class mixup (8b)		
J	I + extra label targets: phase, orientation (step 9)		

Side-by-side comparisons (one choice changed, everything else = best row):

- Mirroring strategy: convert left-handers vs mirror-copy augmentation vs both.
- Scaling: standardised vs unscaled.
- Truncation: keep end vs keep start; length L = 96 / 128 / 160.
- ToF: summaries only vs summaries + 3×3 regions vs full 8×8 images.
- Upside-down subjects: corrected vs removed vs left as-is.

We also run "leave-one-out" checks on the final setup (G minus one step) to show each step still matters once everything else is in.

Baseline models (the brief requires baselines): a simple tree model (LightGBM on per-sequence summary statistics) and a simple 1D-CNN. The modelling teammates own these; we provide the inputs.

10. Team plan and timeline

We have five weeks. A basic but correct pipeline (v1) must reach the modelling teammates by **22 Oct** so they can start training. The physics steps, proof experiments and writing follow.



preprocessing timeline · 5 weekly stages to the 13 Nov deadline

One week per stage, with a hand-off to the modellers at the end of week 2. The split below is a proposal; swap names if you prefer.

Week 1 (9–15 Oct) — both

- [ ] Read this guide fully; note anything unclear as a comment
- [ ] Accept competition rules on Kaggle and download the data into `data/`
- [ ] Install Python packages: pandas, numpy, matplotlib, scikit-learn
- [ ] Run `eda/01_data_audit.py` and fill in Section 11 answers

Week 2 (16–22 Oct) — pipeline v1

- [ ] Steps 1–3: loading, upside-down fix, IMU cleaning (you)
- [ ] Steps 9–12: labels, fixed length + mask, scaling, shared fold file (friend)
- [ ] Step 13: first `preprocess()` + saved files; hand v1 to modellers by 22 Oct

Week 3 (23–29 Oct) — physics and features

- [ ] Steps 4–6: mirroring (all sensors), gravity removal + fallback, smoothing (you)
- [ ] Step 7 features (Section 8) + step 8 ToF/thermopile summaries (friend)
- [ ] Section 8b augmentations as a reusable module (both)
- [ ] Parity test: training path vs Kaggle path give identical outputs

Week 4 (30 Oct–5 Nov) — proof and Kaggle

- [ ] Run the ablation table and side-by-side comparisons (Section 9) with the modellers' baseline
- [ ] Upload saved files to a private Kaggle Dataset; `preprocess()` runs inside the inference notebook; first leaderboard submission
- [ ] Stretch: upside-down detection on test sequences

Week 5 (6–12 Nov) — write-up

- [ ] Write the report's preprocessing section with figures from the audit
- [ ] Record our part of the video (each of us must speak)
- [ ] Submit by **13 Nov, 11:59 pm** — a day early to be safe

11. Things to verify once we run the data audit

These facts decide exact settings; running `eda/01_data_audit.py` on the real files answers each one.

Question	Why it matters	Audit section
Exact row, sequence and subject counts (expect ~8,151 sequences, 81 subjects)	Report numbers; fold sizes	S1–S2
Sequences per gesture (balanced or not?)	Whether we need class weights	S2
Sequence length: median, 90th, 95th percentile (expect mean ~71, max ~700)	Length L for step 10	S3

Question	Why it matters	Audit section
Does `handedness` = 1 mean right or left? Which IMU axes and which ToF/thermopile sensors flip?	Step 4 correctness	S4 + new left-vs-right plot
Do `SUBJ_019262` and `SUBJ_045235` look rotated 180°? Any other odd subject?	Step 2 correctness	New per-subject gravity-direction plot
How many sequences have rotation fully missing? Partly missing?	Size of the fallback in step 5	S5
How often do quaternions flip sign between rows?	Confirms step 3 is needed	New check
% missing per sensor; how many sequences fully IMU-only	Fill strategy; how much dropout	S5
Where the Gesture phase starts (fraction of sequence)	Confirms keep-end truncation	S6
What values `behavior` takes; can we read the initial state from it?	Combined labels in step 9	S6
Quaternion direction: does rotated acceleration put ~9.8 on the vertical axis?	Step 5 correctness	S8
ToF valid range and % of `-1`	The "far" fill and 0–1 scaling	S5, S9
Do thermopiles differ for BFRB vs normal?	Whether thermopile features are worth keeping	S10

12. Glossary

Term	Plain meaning
Ablation	Remove one step, retrain, see how much the score drops — proves that step matters
BFRB	Body-focused repetitive behaviour: hair pulling, skin picking, scratching

Term	Plain meaning
Class imbalance	Some gestures have many more examples than others
Classification	Predicting a category (here: which gesture)
Cross-validation (CV), fold	Split data into 5 parts; train on 4, test on 1, rotate. Each part is a fold
F1 score	One number combining "how often right when we say X" and "how many X's we caught"
Feature	A column the model learns from, raw or calculated
GroupKFold	Cross-validation where one group (here a person) is never split across train and test
IMU	Motion sensor: accelerometer + rotation
Leakage	Accidentally letting the model see information it won't have at test time — gives fake-good scores
Linear acceleration	Acceleration with gravity removed: pure hand motion
NaN	"Not a number" — an empty / missing value
Padding / truncation	Adding zeros to make short sequences longer / cutting long ones shorter
Quaternion	4 numbers describing 3D orientation (tilt + twist)
Sequence	One recorded attempt at a gesture: a group of rows over time
Standardisation	Rescale a column to mean 0, spread 1
Thermopile	Non-contact thermometer
Time-of-Flight (ToF)	Distance sensor that times how long light takes to bounce back; here an 8×8 pixel depth image
World frame	Coordinates of the room (up is always up), as opposed to the watch's own tilted coordinates

Sources

Ideas only; no code copied. Every source we use must be credited in the report.

- Rist — 7th place solution slides (Speaker Deck, Japanese): handedness conversion, upside-down subjects, gravity removal, crops, augmentations, post-processing.
- Top solutions memo, 1st–15th place (Zenn, Japanese): per-placing preprocessing, rotation augmentation angles, label designs.
- Silver-medal solution README (GitHub): gravity removal, angular velocity/distance, ToF aggregation, sensor masking, StratifiedGroupKFold.
- Deep Multimodal Wearable Sensor Fusion for BFRB detection (arXiv 2608.09830): sequence length statistics, ToF scaling, modality ablations.
- Zhou et al. 2019, On the Continuity of Rotation Representations in Neural Networks: why quaternions jump and the 6D rotation format.
- Um et al. 2017, Data Augmentation of Wearable Sensor Data: rotation, time-warping, magnitude-warping augmentations.
- 6th place write-up (Kaggle): upside-down detection at test time (to read in a browser).
- Competition summary (Medium): common patterns across top teams.